

Chapter 20 — Tree DFS

0. What You Will Learn in This Chapter

Tree DFS is one of the most important patterns in coding interviews.

DFS means **Depth-First Search**.

In a tree, DFS explores as deep as possible along one branch before returning and exploring another branch.

Tree DFS is usually implemented recursively because a tree itself is recursive:

tree = root + left subtree + right subtree

By the end of this chapter, you should understand:

- What Tree DFS is
- Why recursion fits trees naturally
- Preorder, inorder, and postorder traversal
- Top-down DFS
- Bottom-up DFS
- DFS with return values
- DFS with global answer
- DFS with path state
- DFS for root-to-leaf problems
- DFS for subtree problems
- DFS for tree height/depth
- DFS for diameter and balance
- DFS for path sum problems
- DFS for lowest common ancestor
- DFS for binary search tree validation
- How to choose traversal order
- How to explain tree recursion clearly

Core problems covered in this chapter:

1. Binary Tree Preorder Traversal
2. Binary Tree Inorder Traversal
3. Binary Tree Postorder Traversal
4. Maximum Depth of Binary Tree
5. Minimum Depth of Binary Tree
6. Same Tree
7. Symmetric Tree
8. Invert Binary Tree
9. Path Sum
10. Path Sum II
11. Binary Tree Paths
12. Diameter of Binary Tree
13. Balanced Binary Tree
14. Count Good Nodes in Binary Tree
15. Validate Binary Search Tree
16. Lowest Common Ancestor of a Binary Tree
17. Binary Tree Maximum Path Sum
18. Path Sum III
19. Subtree of Another Tree
20. Kth Smallest Element in a BST

1. What Is Tree DFS?

Tree DFS is a way to traverse a tree by going deep into one branch before exploring the next branch.

Example tree:

```
  1
 / \
2   3
 / \
4   5
```

DFS can visit nodes in different orders:

Preorder

root -> left -> right

Output:

1, 2, 4, 5, 3

Inorder

left -> root -> right

Output:

4, 2, 5, 1, 3

Postorder

left -> right -> root

Output:

4, 5, 2, 3, 1

2. Why Tree DFS Exists

Many tree problems require information from:

- current node
- left subtree
- right subtree

DFS naturally processes subtrees.

Example:

Maximum depth:

$\text{depth}(\text{root}) = 1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$

This is recursive DFS.

Another example:

Same Tree:

```
same(p, q) =  
  p and q values equal  
  and same(p.left, q.left)  
  and same(p.right, q.right)
```

Again, recursive DFS.

3. Core Need / Why Use Tree DFS Over Other Patterns?

3.1 Why Not Array Traversal?

Trees are not linear.

A node can branch into left and right children.

DFS handles branching naturally.

3.2 Why Not BFS?

BFS is better for level-order problems.

DFS is better for:

- subtree recursion
- path problems
- depth/height
- bottom-up calculations
- ancestor relationships

3.3 Why Not Backtracking?

Tree DFS and backtracking overlap.

If you need all root-to-leaf paths, DFS uses path state and backtracking.

But many tree DFS problems just return values from subtrees.

3.4 Why Not Dynamic Programming?

Some tree problems are tree DP, but basic Tree DFS is the foundation.

Examples:

- House Robber III
 - Binary Tree Maximum Path Sum
-

4. When to Use Tree DFS

Use Tree DFS when the problem asks for:

4.1 Traversal

Examples:

- preorder
- inorder
- postorder

4.2 Depth / Height

Examples:

- Maximum Depth
- Minimum Depth
- Balanced Binary Tree

4.3 Subtree Comparison

Examples:

- Same Tree
- Subtree of Another Tree
- Symmetric Tree

4.4 Path Problems

Examples:

- Path Sum
- Path Sum II
- Binary Tree Paths
- Path Sum III

4.5 Bottom-Up Aggregation

Examples:

- Diameter of Binary Tree
- Maximum Path Sum
- Count Univalued Subtrees

4.6 BST Property

Examples:

- Validate BST
 - Kth Smallest Element in BST
 - Recover BST
-

5. When NOT to Use Tree DFS

5.1 Level Order Required

Use BFS.

Examples:

- Binary Tree Level Order Traversal
- Right Side View
- Zigzag Level Order Traversal

5.2 Shortest Path in Unweighted Graph

Use BFS.

DFS does not guarantee shortest path.

5.3 Tree Is Extremely Deep

Recursive DFS may cause stack overflow.

Use iterative DFS if needed.

5.4 Need Parent Pointers but Not Provided

DFS can build parent map, but if repeated ancestor queries exist, use specialized methods.

6. Core Intuition

At each node, ask:

21. What should I do before visiting children?
22. What information do I need from left subtree?
23. What information do I need from right subtree?
24. What should I return to parent?

25. Do I need a global answer?
This decides traversal style.

7. Mental Models

7.1 Tree as Recursive Object

A tree is made of smaller trees.
So solve left subtree and right subtree recursively.

7.2 Return Information Upward

For height:

child returns height to parent

For balance:

child returns height or failure

For diameter:

child returns height, global answer updates diameter

7.3 Carry Information Downward

For path problems:

parent passes current sum/path/max-so-far to child

Example:

Count Good Nodes passes max value seen so far.

7.4 Global Answer

Some problems need answer across all nodes, not just root.

Example:

Diameter.

Each node can be center of best path.

Use global variable.

8. DFS Traversal Types

8.1 Preorder

Process root before children.

root -> left -> right

Use when:

- copying tree
- serializing tree
- top-down state
- path building

8.2 Inorder

Process left, root, right.

left -> root -> right

Use when:

- BST sorted order
- kth smallest in BST
- validate BST with sorted property

8.3 Postorder

Process children before root.

left -> right -> root

Use when:

- deleting tree
- height
- diameter
- balance
- subtree aggregation

9. C++ Tools Needed

9.1 TreeNode Definition

```
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode() : val(0), left(nullptr), right(nullptr) {}

    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}

    TreeNode(int x, TreeNode* left, TreeNode* right)
        : val(x), left(left), right(right) {}
};
```

9.2 Null Check

Always handle:

if (root == nullptr)

before accessing:

```
root->val
root->left
root->right
```

9.3 Result Vector

```
vector<int> result;
```

9.4 Path Vector

```
vector<int> path;
```

Remember to pop after DFS.

10. Core Templates

Template 1 — Preorder DFS

```
void preorder(TreeNode* root, vector<int>& result) {
    if (root == nullptr) {
        return;
    }

    result.push_back(root->val);

    preorder(root->left, result);
    preorder(root->right, result);
}
```

Template 2 — Inorder DFS

```
void inorder(TreeNode* root, vector<int>& result) {
    if (root == nullptr) {
        return;
    }

    inorder(root->left, result);

    result.push_back(root->val);

    inorder(root->right, result);
}
```

Template 3 — Postorder DFS

```
void postorder(TreeNode* root, vector<int>& result) {
    if (root == nullptr) {
        return;
    }

    postorder(root->left, result);
    postorder(root->right, result);

    result.push_back(root->val);
}
```

Template 4 — Return Height

```
int height(TreeNode* root) {  
    if (root == nullptr) {  
        return 0;  
    }  
  
    int left = height(root->left);  
    int right = height(root->right);  
  
    return 1 + max(left, right);  
}
```

Template 5 — Top-Down Path State

```
void dfs(TreeNode* root, int currentSum) {  
    if (root == nullptr) {  
        return;  
    }  
  
    currentSum += root->val;  
  
    dfs(root->left, currentSum);  
    dfs(root->right, currentSum);  
}
```

Template 6 — Path Backtracking

```
void dfs(TreeNode* root, vector<int>& path, vector<vector<int>>& result) {  
    if (root == nullptr) {  
        return;  
    }  
  
    path.push_back(root->val);  
  
    if (root->left == nullptr && root->right == nullptr) {  
        result.push_back(path);  
    }  
  
    dfs(root->left, path, result);  
    dfs(root->right, path, result);  
}
```

```
    path.pop_back();  
}
```

Template 7 — Global Answer Bottom-Up

```
int answer = 0;  
  
int dfs(TreeNode* root) {  
    if (root == nullptr) {  
        return 0;  
    }  
  
    int left = dfs(root->left);  
    int right = dfs(root->right);  
  
    answer = max(answer, left + right);  
  
    return 1 + max(left, right);  
}
```

Template 8 — BST Inorder

```
void inorder(TreeNode* root) {  
    if (root == nullptr) {  
        return;  
    }  
  
    inorder(root->left);  
  
    // process root->val in sorted order  
  
    inorder(root->right);  
}
```

11. Major Variations of Tree DFS

Preorder	root before children	Binary Tree Paths
Inorder	left-root-right	Kth Smallest BST
Postorder	children before root	Max Depth, Diameter

Variation	Direction	Example
Top-down DFS	pass state downward	Path Sum
Bottom-up DFS	return info upward	Balanced Tree
Global answer DFS	update external answer	Diameter
Path backtracking	push/pop path	Path Sum II
Mirror recursion	compare opposite children	Symmetric Tree
BST recursion	value bounds	Validate BST

12. Full Problem Walkthrough 1 — LeetCode 144: Binary Tree Preorder Traversal

12.1 Problem Summary

Return preorder traversal of binary tree.

Order:

root, left, right

12.2 How to Identify This Pattern

Problem directly asks for preorder traversal.

Use recursive DFS.

12.3 C++ Code

```
class Solution {
public:
    void dfs(TreeNode* root, vector<int>& result) {
        if (root == nullptr) {
            return;
        }

        result.push_back(root->val);

        dfs(root->left, result);
        dfs(root->right, result);
    }

    vector<int> preorderTraversal(TreeNode* root) {
```

```
vector<int> result;

dfs(root, result);

return result;
}
};
```

12.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

where h is tree height.

13. Full Problem Walkthrough 2 — LeetCode 94: Binary Tree Inorder Traversal

13.1 Problem Summary

Return inorder traversal.

Order:

left, root, right

13.2 How to Identify This Pattern

Problem asks for inorder.

In BST, inorder gives sorted values.

13.3 C++ Code

```
class Solution {
public:
    void dfs(TreeNode* root, vector<int>& result) {
        if (root == nullptr) {
            return;
        }

        dfs(root->left, result);
```

```
        result.push_back(root->val);

        dfs(root->right, result);
    }

    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> result;

        dfs(root, result);

        return result;
    }
};
```

13.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

14. Full Problem Walkthrough 3 — LeetCode 145: Binary Tree Postorder Traversal

14.1 Problem Summary

Return postorder traversal.

Order:

left, right, root

14.2 How to Identify This Pattern

Postorder is useful when children must be processed before parent.

14.3 C++ Code

```
class Solution {
public:
    void dfs(TreeNode* root, vector<int>& result) {
        if (root == nullptr) {
            return;
        }
    }
};
```

```

    }

    dfs(root->left, result);
    dfs(root->right, result);

    result.push_back(root->val);
}

vector<int> postorderTraversal(TreeNode* root) {
    vector<int> result;

    dfs(root, result);

    return result;
}
};

```

14.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

15. Full Problem Walkthrough 4 — LeetCode 104: Maximum Depth of Binary Tree

15.1 Problem Summary

Return maximum depth of binary tree.

Depth means number of nodes along longest root-to-leaf path.

15.2 How to Identify This Pattern

Need height/depth.

Use postorder DFS:

```
depth = 1 + max(leftDepth, rightDepth)
```

15.3 C++ Code

```
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }

        int left = maxDepth(root->left);
        int right = maxDepth(root->right);

        return 1 + max(left, right);
    }
};
```

15.4 Dry Run

Tree:

```
  1
 / \
2   3
/
4
```

Depth of node 4 is 1.
Depth of node 2 is 2.
Depth of root 1 is 3.

15.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

16. Full Problem Walkthrough 5 — LeetCode 111: Minimum Depth of Binary Tree

16.1 Problem Summary

Return minimum depth from root to nearest leaf.
A leaf has no children.

16.2 Important Trap

If one child is null, do not take min with 0.

Example:

```
1
 \
 2
```

Minimum depth is 2, not 1.

16.3 C++ Code

```
class Solution {
public:
    int minDepth(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }

        if (root->left == nullptr && root->right == nullptr) {
            return 1;
        }

        if (root->left == nullptr) {
            return 1 + minDepth(root->right);
        }

        if (root->right == nullptr) {
            return 1 + minDepth(root->left);
        }

        return 1 + min(minDepth(root->left), minDepth(root->right));
    }
};
```

16.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

16.5 Common Mistake

Wrong:

```
return 1 + min(minDepth(root->left), minDepth(root->right));
```

This fails when one child is null.

17. Full Problem Walkthrough 6 — LeetCode 100: Same Tree

17.1 Problem Summary

Return true if two binary trees are structurally identical and values match.

17.2 How to Identify This Pattern

Need compare two trees recursively.

17.3 C++ Code

```
class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if (p == nullptr && q == nullptr) {
            return true;
        }

        if (p == nullptr || q == nullptr) {
            return false;
        }

        if (p->val != q->val) {
            return false;
        }

        return isSameTree(p->left, q->left) &&
            isSameTree(p->right, q->right);
    }
};
```

17.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

18. Full Problem Walkthrough 7 — LeetCode 101: Symmetric Tree

18.1 Problem Summary

Return true if a binary tree is symmetric around its center.

18.2 How to Identify This Pattern

Need mirror comparison.

Left subtree and right subtree must mirror each other.

18.3 C++ Code

```
class Solution {
public:
    bool mirror(TreeNode* a, TreeNode* b) {
        if (a == nullptr && b == nullptr) {
            return true;
        }

        if (a == nullptr || b == nullptr) {
            return false;
        }

        if (a->val != b->val) {
            return false;
        }

        return mirror(a->left, b->right) &&
            mirror(a->right, b->left);
    }

    bool isSymmetric(TreeNode* root) {
        if (root == nullptr) {
            return true;
        }

        return mirror(root->left, root->right);
    }
};
```

18.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

19. Full Problem Walkthrough 8 — LeetCode 226: Invert Binary Tree

19.1 Problem Summary

Invert binary tree by swapping left and right children recursively.

19.2 How to Identify This Pattern

Need modify every node.

Use DFS and swap children.

19.3 C++ Code

```
class Solution {  
public:  
    TreeNode* invertTree(TreeNode* root) {  
        if (root == nullptr) {  
            return nullptr;  
        }  
  
        TreeNode* left = invertTree(root->left);  
        TreeNode* right = invertTree(root->right);  
  
        root->left = right;  
        root->right = left;  
  
        return root;  
    }  
};
```

19.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

20. Full Problem Walkthrough 9 — LeetCode 112: Path Sum

20.1 Problem Summary

Return true if there is a root-to-leaf path whose sum equals target.

20.2 How to Identify This Pattern

Need carry remaining target down the tree.

This is top-down DFS.

20.3 C++ Code

```
class Solution {
public:
    bool hasPathSum(TreeNode* root, int targetSum) {
        if (root == nullptr) {
            return false;
        }

        if (root->left == nullptr && root->right == nullptr) {
            return targetSum == root->val;
        }

        int remaining = targetSum - root->val;

        return hasPathSum(root->left, remaining) ||
            hasPathSum(root->right, remaining);
    }
};
```

20.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

21. Full Problem Walkthrough 10 — LeetCode 113: Path Sum II

21.1 Problem Summary

Return all root-to-leaf paths where sum equals target.

21.2 How to Identify This Pattern

Need collect all paths.

Use DFS with path vector and backtracking.

21.3 C++ Code

```
class Solution {
public:
    void dfs(TreeNode* root, int targetSum,
              vector<int>& path,
              vector<vector<int>>& result) {
        if (root == nullptr) {
            return;
        }

        path.push_back(root->val);

        if (root->left == nullptr && root->right == nullptr &&
            targetSum == root->val) {
            result.push_back(path);
        }

        dfs(root->left, targetSum - root->val, path, result);
        dfs(root->right, targetSum - root->val, path, result);

        path.pop_back();
    }

    vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
        vector<vector<int>> result;
        vector<int> path;

        dfs(root, targetSum, path, result);

        return result;
    }
};
```

21.4 Complexity

Time:

$O(n * h)$

in worst case due to copying paths.

Space:

$O(h)$

excluding output.

21.5 Common Mistakes

- Forgetting `path.pop_back()`
- Adding non-leaf paths
- Not subtracting current node value

22. Full Problem Walkthrough 11 — LeetCode 257: Binary Tree Paths

22.1 Problem Summary

Return all root-to-leaf paths as strings.

Example:

1->2->5

1->3

22.2 How to Identify This Pattern

Need all root-to-leaf paths.

Use DFS with path string.

22.3 C++ Code

```
class Solution {
public:
    void dfs(TreeNode* root, string path, vector<string>& result) {
        if (root == nullptr) {
            return;
        }

        if (!path.empty()) {
            path += "->";
        }

        path += to_string(root->val);

        if (root->left == nullptr && root->right == nullptr) {
            result.push_back(path);
            return;
        }
    }
};
```

```

        dfs(root->left, path, result);
        dfs(root->right, path, result);
    }

    vector<string> binaryTreePaths(TreeNode* root) {
        vector<string> result;

        dfs(root, "", result);

        return result;
    }
};

```

22.4 Complexity

Time:

$O(n * h)$

because path strings are copied.

Space:

$O(h)$

recursion stack.

23. Full Problem Walkthrough 12 — LeetCode 543: Diameter of Binary Tree

23.1 Problem Summary

Diameter is length of longest path between any two nodes.

Path may or may not pass through root.

Return number of edges.

23.2 How to Identify This Pattern

For each node, possible diameter through it is:

left height + right height

Need compute height bottom-up and update global answer.

23.3 C++ Code

```

class Solution {
public:

```

```
int answer = 0;

int height(TreeNode* root) {
    if (root == nullptr) {
        return 0;
    }

    int left = height(root->left);
    int right = height(root->right);

    answer = max(answer, left + right);

    return 1 + max(left, right);
}

int diameterOfBinaryTree(TreeNode* root) {
    height(root);
    return answer;
}
};
```

23.4 Dry Run Concept

At each node:

- left height = longest downward path in left subtree
- right height = longest downward path in right subtree
- path through node uses both sides

Update answer.

23.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

23.6 Common Mistake

Computing height repeatedly for every node leads to $O(n^2)$.

Correct approach computes height once per node.

24. Full Problem Walkthrough 13 —

LeetCode 110: Balanced Binary Tree

24.1 Problem Summary

Return true if tree is height-balanced.

Balanced means for every node:

$$\text{abs}(\text{height}(\text{left}) - \text{height}(\text{right})) \leq 1$$

24.2 How to Identify This Pattern

Need height and balance from children.

Use bottom-up DFS.

Return -1 if subtree is unbalanced.

24.3 C++ Code

```
class Solution {
public:
    int check(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }

        int left = check(root->left);

        if (left == -1) {
            return -1;
        }

        int right = check(root->right);

        if (right == -1) {
            return -1;
        }

        if (abs(left - right) > 1) {
            return -1;
        }

        return 1 + max(left, right);
    }
}
```

```
bool isBalanced(TreeNode* root) {  
    return check(root) != -1;  
}  
};
```

24.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

25. Full Problem Walkthrough 14 — LeetCode 1448: Count Good Nodes in Binary Tree

25.1 Problem Summary

A node is good if no node on path from root to it has value greater than it.
Return number of good nodes.

25.2 How to Identify This Pattern

Need carry maximum value seen so far down the path.
This is top-down DFS.

25.3 C++ Code

```
class Solution {  
public:  
    int dfs(TreeNode* root, int maxSoFar) {  
        if (root == nullptr) {  
            return 0;  
        }  
  
        int good = 0;  
  
        if (root->val >= maxSoFar) {  
            good = 1;  
        }  
  
        maxSoFar = max(maxSoFar, root->val);  
    }  
};
```

```
        return good +
            dfs(root->left, maxSoFar) +
            dfs(root->right, maxSoFar);
    }

    int goodNodes(TreeNode* root) {
        return dfs(root, INT_MIN);
    }
};
```

25.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

26. Full Problem Walkthrough 15 — LeetCode 98: Validate Binary Search Tree

26.1 Problem Summary

Return true if binary tree is valid BST.

BST rule:

- all values in left subtree must be less than node
- all values in right subtree must be greater than node

26.2 How to Identify This Pattern

Need validate range constraints.

DFS with lower and upper bounds.

26.3 C++ Code

```
class Solution {
public:
    bool validate(TreeNode* root, long long low, long long high) {
        if (root == nullptr) {
            return true;
        }
    }
```

```

    if (root->val <= low || root->val >= high) {
        return false;
    }

    return validate(root->left, low, root->val) &&
        validate(root->right, root->val, high);
}

bool isValidBST(TreeNode* root) {
    return validate(root, LLONG_MIN, LLONG_MAX);
}
};

```

26.4 Why Not Only Compare With Children?

Wrong:

```

root->left->val < root->val
root->right->val > root->val

```

This misses deeper violations.

Example:

```

  5
 / \
1   6
 / \
4   7

```

4 is in right subtree of 5, so invalid.

26.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

27. Full Problem Walkthrough 16 — LeetCode 236: Lowest Common Ancestor of a Binary Tree

27.1 Problem Summary

Given binary tree and two nodes p and q , return their lowest common ancestor.

27.2 How to Identify This Pattern

Need find node where p and q split into different subtrees.

Use postorder DFS.

27.3 C++ Code

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p,
    TreeNode* q) {
        if (root == nullptr || root == p || root == q) {
            return root;
        }

        TreeNode* left = lowestCommonAncestor(root->left, p, q);
        TreeNode* right = lowestCommonAncestor(root->right, p, q);

        if (left != nullptr && right != nullptr) {
            return root;
        }

        return left != nullptr ? left : right;
    }
};
```

27.4 Explanation

If one node found in left subtree and one in right subtree, current root is LCA.

If both are in one side, return that side's result.

27.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

28. Full Problem Walkthrough 17 — LeetCode 124: Binary Tree Maximum Path Sum

28.1 Problem Summary

A path can start and end at any nodes.

Return maximum path sum.

Path must be connected and can move parent-child.

28.2 How to Identify This Pattern

For each node, max path through it:

node value + best left gain + best right gain

But return to parent only one side:

node value + max(left gain, right gain)

Use bottom-up DFS with global answer.

28.3 C++ Code

```
class Solution {
public:
    int answer = INT_MIN;

    int gain(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }

        int left = max(0, gain(root->left));
        int right = max(0, gain(root->right));

        int pathThroughRoot = root->val + left + right;

        answer = max(answer, pathThroughRoot);

        return root->val + max(left, right);
    }

    int maxPathSum(TreeNode* root) {
        gain(root);

        return answer;
    }
};
```

28.4 Why Use max(0, gain(...))?

If a subtree contributes negative sum, ignore it.

Better not to include negative path.

28.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

28.6 Common Mistake

Returning both left and right to parent.

A path going upward can only choose one branch, otherwise it would fork.

29. Full Problem Walkthrough 18 — LeetCode 437: Path Sum III

29.1 Problem Summary

Count number of paths whose sum equals target.

Path can start and end anywhere, but must go downward.

29.2 How to Identify This Pattern

This is tree DFS + prefix sum.

Current path sum from root to current node is prefix.

Need previous prefix:

$\text{currentPrefix} - \text{target}$

Use hash map of prefix counts.

29.3 C++ Code

```
class Solution {
public:
    int dfs(TreeNode* root, long long prefix, int target,
            unordered_map<long long, int>& count) {
        if (root == nullptr) {
            return 0;
        }

        prefix += root->val;

        int answer = 0;

        if (count.count(prefix - target)) {
```

```

        answer += count[prefix - target];
    }

    count[prefix]++;

    answer += dfs(root->left, prefix, target, count);
    answer += dfs(root->right, prefix, target, count);

    count[prefix]--;

    return answer;
}

int pathSum(TreeNode* root, int targetSum) {
    unordered_map<long long, int> count;
    count[0] = 1;

    return dfs(root, 0, targetSum, count);
}
};

```

29.4 Why Decrement Count?

Prefix map represents current root-to-node path only.
 When returning from a node, remove its prefix before exploring other branches.
 This is backtracking.

29.5 Complexity

Time:

$O(n)$

Space:

$O(h)$

plus hash map for current path.

30. Full Problem Walkthrough 19 — LeetCode 572: Subtree of Another Tree

30.1 Problem Summary

Return true if subRoot is a subtree of root.

30.2 How to Identify This Pattern

Need compare trees at every node.

Use DFS:

- if current tree same as subRoot, true
 - else check left and right
-

30.3 C++ Code

```
class Solution {
public:
    bool same(TreeNode* a, TreeNode* b) {
        if (a == nullptr && b == nullptr) {
            return true;
        }

        if (a == nullptr || b == nullptr) {
            return false;
        }

        if (a->val != b->val) {
            return false;
        }

        return same(a->left, b->left) &&
               same(a->right, b->right);
    }

    bool isSubtree(TreeNode* root, TreeNode* subRoot) {
        if (root == nullptr) {
            return false;
        }

        if (same(root, subRoot)) {
            return true;
        }

        return isSubtree(root->left, subRoot) ||
               isSubtree(root->right, subRoot);
    }
};
```

30.4 Complexity

Worst-case:

$O(n * m)$

where n is root size and m is subRoot size.

31. Full Problem Walkthrough 20 — LeetCode 230: Kth Smallest Element in a BST

31.1 Problem Summary

Given BST root and integer k , return k th smallest value.

31.2 How to Identify This Pattern

BST inorder traversal gives sorted order.

Need k th element in inorder.

31.3 C++ Code

```
class Solution {
public:
    int answer = -1;
    int count = 0;

    void inorder(TreeNode* root, int k) {
        if (root == nullptr) {
            return;
        }

        inorder(root->left, k);

        count++;

        if (count == k) {
            answer = root->val;
            return;
        }

        inorder(root->right, k);
    }

    int kthSmallest(TreeNode* root, int k) {
        inorder(root, k);
    }
};
```

```
        return answer;
    }
};
```

31.4 Complexity

Time:

$O(h + k)$

average if early stopped effectively.

Space:

$O(h)$

32. Mini Problem 1 — Count Nodes in Binary Tree

```
int countNodes(TreeNode* root) {
    if (root == nullptr) {
        return 0;
    }

    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

33. Mini Problem 2 — Sum of Left Leaves

```
class Solution {
public:
    int sumOfLeftLeaves(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }

        int sum = 0;

        if (root->left != nullptr &&
            root->left->left == nullptr &&
            root->left->right == nullptr) {
```

```

        sum += root->left->val;
    }

    sum += sumOfLeftLeaves(root->left);
    sum += sumOfLeftLeaves(root->right);

    return sum;
}
};

```

34. Mini Problem 3 — Binary Tree Tilt

```

class Solution {
public:
    int totalTilt = 0;

    int subtreeSum(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }

        int left = subtreeSum(root->left);
        int right = subtreeSum(root->right);

        totalTilt += abs(left - right);

        return root->val + left + right;
    }

    int findTilt(TreeNode* root) {
        subtreeSum(root);
        return totalTilt;
    }
};

```

35. Mini Problem 4 — Root-to-Leaf Sum Numbers

```

class Solution {
public:
    int dfs(TreeNode* root, int current) {
        if (root == nullptr) {
            return 0;
        }

        current = current * 10 + root->val;

        if (root->left == nullptr && root->right == nullptr) {
            return current;
        }

        return dfs(root->left, current) + dfs(root->right, current);
    }

    int sumNumbers(TreeNode* root) {
        return dfs(root, 0);
    }
};

```

36. Common Edge Cases for Tree DFS

36.1 Empty Tree

Return depends on problem:

- depth: 0
- same tree: true if both empty
- path sum: false
- traversal: empty vector

36.2 Single Node

Often base case.

36.3 Skewed Tree

Recursion depth becomes $O(n)$.

36.4 Negative Values

Important for:

- maximum path sum
- path sum
- BST bounds

36.5 Leaf Detection

Leaf means:

```
root->left == nullptr && root->right == nullptr
```

36.6 Global Answer Initialization

For maximum path sum, initialize with:

```
INT_MIN
```

not 0, because all values may be negative.

37. Common Mistakes and Wrong Code

Mistake 1 — Accessing Null Root

Wrong:

```
if (root->left == nullptr)
```

before checking root.

Correct:

```
if (root == nullptr)
```

Mistake 2 — Wrong Minimum Depth Formula

Wrong:

```
1 + min(leftDepth, rightDepth)
```

when one child is null.

Mistake 3 — Forgetting Path Pop

For path collection:

```
path.push_back(root->val);
```

```
...
```

```
path.pop_back();
```

Mistake 4 — Returning Global Answer Instead of Local Gain

For max path sum, return one-side gain, not full path.

Mistake 5 — Validating BST Only Locally

Need bounds from ancestors, not just direct children.

Mistake 6 — Not Backtracking Prefix Sum Map

Path Sum III needs decrement after DFS.

38. Interview Explanation Templates

38.1 Tree Depth

I recursively compute the depth of left and right subtrees. The current node's depth is one plus the maximum of those two depths.

38.2 Path Sum

I carry the remaining target down the tree. At a leaf, I check whether the remaining target equals the leaf value.

38.3 Diameter

At each node, I compute left and right heights. The path through that node has length left height plus right height. I update a global answer and return the height to the parent.

38.4 Validate BST

Each node has an allowed range based on its ancestors. Left children must be smaller than the current node, and right children must be larger. I pass updated bounds recursively.

38.5 Lowest Common Ancestor

If one target is found in the left subtree and the other in the right subtree, the current node is the LCA. Otherwise, return whichever side contains a target.

39. Complexity Patterns

Traversal	$O(n)$	$O(h)$
Max depth	$O(n)$	$O(h)$
Same tree	$O(n)$	$O(h)$
Symmetric tree	$O(n)$	$O(h)$
Path Sum	$O(n)$	$O(h)$
Path Sum II	$O(n * h)$ worst	$O(h)$
Diameter	$O(n)$	$O(h)$
Balanced tree	$O(n)$	$O(h)$
Validate BST	$O(n)$	$O(h)$
LCA	$O(n)$	$O(h)$
Max Path Sum	$O(n)$	$O(h)$
Path Sum III	$O(n)$	$O(h)$
Subtree of Another Tree	$O(n * m)$ worst	$O(h)$
Kth Smallest BST	$O(h + k)$	$O(h)$

40. Time-Boxed Practice Guidance

Easy Tree DFS

Target:

10–20 minutes

Examples:

- Maximum Depth
- Same Tree
- Invert Binary Tree
- Binary Tree Paths

Medium Tree DFS

Target:

25–45 minutes

Examples:

- Path Sum II
- Diameter
- Balanced Binary Tree
- Validate BST
- Lowest Common Ancestor

Harder Tree DFS

Target:

45–75 minutes

Examples:

- Binary Tree Maximum Path Sum
- Path Sum III
- Subtree of Another Tree
- Recover BST
- House Robber III

41. Expanded Practice Set

41.1 Must-Do Easy

26. Binary Tree Preorder Traversal
27. Binary Tree Inorder Traversal
28. Binary Tree Postorder Traversal
29. Maximum Depth of Binary Tree
30. Minimum Depth of Binary Tree
31. Same Tree
32. Symmetric Tree
33. Invert Binary Tree
34. Path Sum
35. Binary Tree Paths

41.2 Must-Do Medium

36. Path Sum II
37. Diameter of Binary Tree

- 38. Balanced Binary Tree
- 39. Count Good Nodes in Binary Tree
- 40. Validate Binary Search Tree
- 41. Lowest Common Ancestor of a Binary Tree
- 42. Kth Smallest Element in a BST
- 43. Subtree of Another Tree
- 44. Sum Root to Leaf Numbers
- 45. Binary Tree Tilt

41.3 Strong Interview Practice

- 46. Binary Tree Maximum Path Sum
- 47. Path Sum III
- 48. Construct Binary Tree from Preorder and Inorder Traversal
- 49. Construct Binary Tree from Inorder and Postorder Traversal
- 50. Serialize and Deserialize Binary Tree
- 51. House Robber III
- 52. Flatten Binary Tree to Linked List
- 53. Recover Binary Search Tree

41.4 Hard / Stretch

- 54. Binary Tree Maximum Path Sum
 - 55. Serialize and Deserialize Binary Tree
 - 56. Recover Binary Search Tree
 - 57. House Robber III
 - 58. Count of Smaller Numbers After Self, BST/Fenwick alternative
 - 59. Number of Ways to Reorder Array to Get Same BST
-

42. Suggested Study Order

Step 1 — Basic Traversals

Start with:

- 60. Preorder Traversal
- 61. Inorder Traversal
- 62. Postorder Traversal

Goal:

Understand DFS order.

Step 2 — Basic Recursive Return

Solve:

- 63. Maximum Depth
- 64. Minimum Depth
- 65. Count Nodes
- 66. Invert Binary Tree

Goal:

Learn returning values from subtrees.

Step 3 — Tree Comparison

Solve:

- 67. Same Tree
- 68. Symmetric Tree

69. Subtree of Another Tree

Goal:

Learn recursive comparison.

Step 4 — Path Problems

Solve:

- 70. Path Sum
- 71. Path Sum II
- 72. Binary Tree Paths
- 73. Sum Root to Leaf Numbers

Goal:

Learn carrying path state.

Step 5 — Bottom-Up Global Answer

Solve:

- 74. Diameter
- 75. Balanced Tree
- 76. Maximum Path Sum
- 77. Binary Tree Tilt

Goal:

Learn return local info, update global answer.

Step 6 — BST DFS

Solve:

- 78. Validate BST
- 79. Kth Smallest in BST
- 80. LCA of Binary Tree

Goal:

Use inorder, bounds, and ancestor logic.

43. Pattern Decision Table

root-left-right	preorder	Preorder Traversal
left-root-right	inorder	BST sorted order
left-right-root	postorder	Height / Diameter
maximum depth	bottom-up return	Max Depth
minimum depth	leaf-aware DFS	Min Depth
compare two trees	paired recursion	Same Tree
mirror tree	opposite recursion	Symmetric Tree
root-to-leaf sum	top-down target	Path Sum

Problem Clue	DFS Type	Example
collect all paths	path backtracking	Path Sum II
longest path	global answer + height	Diameter
balanced check	return height or -1	Balanced Tree
ancestor relation	postorder target return	LCA
BST validity	bounds recursion	Validate BST
kth in BST	inorder count	Kth Smallest
any downward path sum	prefix sum map	Path Sum III

44. Final Cheat Sheet

Preorder

```
result.push_back(root->val);
dfs(root->left);
dfs(root->right);
```

Inorder

```
dfs(root->left);
result.push_back(root->val);
dfs(root->right);
```

Postorder

```
dfs(root->left);
dfs(root->right);
result.push_back(root->val);
```

Height

```
if (root == nullptr) {
    return 0;
}

return 1 + max(height(root->left), height(root->right));
```

Path Backtracking

```
path.push_back(root->val);

dfs(root->left);
dfs(root->right);
```

```
path.pop_back();
```

Diameter

```
int left = height(root->left);  
int right = height(root->right);  
  
answer = max(answer, left + right);  
  
return 1 + max(left, right);
```

Validate BST

```
bool validate(TreeNode* root, long long low, long long high) {  
    if (root == nullptr) {  
        return true;  
    }  
  
    if (root->val <= low || root->val >= high) {  
        return false;  
    }  
  
    return validate(root->left, low, root->val) &&  
        validate(root->right, root->val, high);  
}
```

LCA

```
if (root == nullptr || root == p || root == q) {  
    return root;  
}  
  
TreeNode* left = dfs(root->left);  
TreeNode* right = dfs(root->right);  
  
if (left != nullptr && right != nullptr) {  
    return root;  
}  
  
return left != nullptr ? left : right;
```

45. What to Master Before Chapter 21

Before moving to Tree BFS, make sure you can:

- Write preorder traversal
 - Write inorder traversal
 - Write postorder traversal
 - Explain DFS recursion on trees
 - Solve Maximum Depth
 - Solve Same Tree
 - Solve Symmetric Tree
 - Solve Invert Binary Tree
 - Solve Path Sum
 - Use path vector with push/pop
 - Solve Diameter using global answer
 - Solve Balanced Binary Tree bottom-up
 - Validate BST using bounds
 - Solve LCA recursively
 - Understand when to use top-down vs bottom-up DFS
 - Handle null root safely
-

46. Core Takeaway

Tree DFS is recursion on tree structure.

The key question is:

What should each node return to its parent?

or:

What state should each node receive from its parent?

Use:

- preorder when parent state matters before children
- inorder for BST sorted order
- postorder when children information is needed first
- global answer when best result may occur anywhere

Tree DFS becomes easy when you clearly define:

base case

work at current node

recursive calls

return value

global/path state if needed